

Arab Open University – Bahrain

Faculty of Computer Studies

Smart Inventory Tracking

Inventory Management for Satellite Store at AlBa

Student ID: 6220354

Name: Maheen Muhammad Zafeer

Supervisor: Dr. Mohammad Riyaz Belgaum

Smart Inventory Tracking

Satellite Store

AlBa

~ Introduction ~

- ❖ Industrial maintenance stores manage thousands of inventory items daily

- ❖ Many still rely on spreadsheets or handwritten logs

 - ❖ Manual methods are slow and error-prone

 - ❖ Lack of real-time visibility causes shortages and overstocking

- ❖ Aim:

 - ❖ Develop a Smart Inventory Tracking System to automate inventory tracking and improve stock visibility in real time

 - ❖ Monitor material movement and stock levels

 - ❖ Improve accuracy, efficiency and visibility of store inventory

 - ❖ Support better planning and cost reduction

~ Suggested Solution ~

❖ Smart Inventory Tracking System Mobile Application with barcode scanning

- ❖ Central SQL database for accurate transaction logging
- ❖ Real-time stock updates for withdrawals and restocking
- ❖ Automated alerts for low or overstock situations
- ❖ Dashboard with usage trends
- ❖ Simple, user-friendly interface with minimal IT dependency

❖ Requirements Gathering:

- ❖ Meeting & emails with AlBa Supervisor
- ❖ Observation of current Excel-based System
- ❖ Review of similar inventory systems

~ Aims ~

- ❖ The main aim of this project is to develop a Smart Inventory Tracking System that automates tracking, provides real-time stock visibility and improves planning and availability of essential materials.
- ❖ Specific Objectives:
 - ❖ Maintain accurate inventory levels through real-time stock withdrawal and restocking updates.
 - ❖ Generate low-stock alerts for items below low-threshold and overstock alerts for items being restocked over the high threshold, preventing shortages or overstocking.
 - ❖ Provide an analytical dashboard that displays key performance indicators and usage trends for better planning.
 - ❖ Maintain an auditable history of inventory transactions for accountability and reporting.

~ Requirements ~

Functional Requirements

- ❖ Barcode-based stock withdrawal and restocking
- ❖ Central SQL database for all records
- ❖ Transaction logging
- ❖ Real-time update of inventory levels
- ❖ Dashboard and reports for usage trends
- ❖ Low & high stock alerts
- ❖ Add, edit and manage item categories
- ❖ Search by item name or category
- ❖ Order management, scheduled maintenance reservations and repair tracking

Non-Functional Requirements

- ❖ Usability: Simple interface for non-technical staff
- ❖ Performance: Real-time updates and fast processing
- ❖ Reliability: Accurate stock synchronisation
- ❖ Auditability: All actions fully traceable
- ❖ Security: Protected role-based access
- ❖ Scalability: Future multi-store expansion
- ❖ Maintainability: Easy to update and extend

~ Database Design~

TRANSACTION_LOG	
Transaction_ID (PK) :	String
Transaction_Quantity :	Int
Material_ID (FK) :	String
User_ID (FK) :	String
Transaction_Type :	String
Transaction_Time :	Date/Time
Order_ID (FK) :	String
Job_ID (FK) :	String

SCHEDULED_MAINTENANCE	
Maintenance_ID (PK) :	String
Job_ID (FK) :	String
Scheduled_At :	Date/Time
Scheduled_For :	Date/Time
User_ID (FK) :	String
Status :	String
Updated_By (FK) :	String

USER	
User_ID (PK) :	String
Username :	String
Full_Name :	String
Email :	String
Phone_Number :	String
Role :	String
Created_At :	Date/Time

JOB	
Job_ID (PK) :	String
Maintenance_Role :	String
Position :	String

REPAIR	
Material_ID (FK) :	String
Repair_Status :	String
Repair_Quantity :	Int

MATERIAL	
Material_ID (PK) :	String
Barcode_ID (PK) :	String
Current_Quantity :	Int
Material_Description :	String
Category :	String
Low_Threshold :	Int
High_Threshold :	Int
Item_Type :	String

SCHEDULED_MATERIALS	
Maintenance_ID (FK) :	String
Material_ID (FK) :	String
Quantity :	Int

ORDER_MATERIALS	
Order_ID (FK) :	String
Material_ID (FK) :	String
Material_Qty :	Int

ORDER	
Order_ID (PK) :	String
User_ID (FK) :	String
Order_Date :	Date
Order_Status :	String
Updated_By (FK) :	String

~ Implementation & Testing ~

❖ Tools

- ❖ Framework: Flutter / Dart
- ❖ Database: Supabase PostgreSQL
- ❖ Barcode Scanner: Mobile Scanner Library
- ❖ Analytics: fl_chart
- ❖ IDE: Visual Studio Code

❖ Secure user authentication using Supabase – login and signup pages with input validation.

❖ The testing methods used are functional testing and user acceptance testing.

- ❖ Functional testing verifies that the software performs specified functions correctly and ensures it meets requirements.
- ❖ User acceptance testing checks whether the software meets user needs and confirms it is ready for real-world use.

~ Results & Discussions ~

Results

- ❖ Barcode scanning improved speed and accuracy
- ❖ Real-time updates improved stock visibility
- ❖ Validation checks improved data reliability
- ❖ Reports enabled better stock planning
- ❖ Role-based access improved security and usability

Goals Achieved

- ❖ Successfully automated stock tracking
- ❖ Implemented alerts and warnings
- ❖ Built dashboard and analytics reports
- ❖ Maintained full transaction audit history
- ❖ PDF export



~ Results & Discussions ~

Future Improvements

- ❖ Push notifications / Email alerts
- ❖ Offline functionality
- ❖ Advanced predictive analytics
- ❖ An admin dashboard to approve orders before placing them
- ❖ Calendar view for maintenance schedules
- ❖ Supplier integration
- ❖ Multi-store support
- ❖ Web/Desktop version

~ Conclusion ~

- ❖ The Smart Inventory Tracking System successfully replaces manual and spreadsheet-based inventory tracking with a mobile-based digital solution.
 - ❖ It improves accuracy, efficiency and real-time visibility of stock movement in a maintenance store environment.
 - ❖ Core objectives were achieved including real-time stock updates, barcode scanning, automated alerts and transaction logging.
 - ❖ Role-based access control and validation checks improved system security and reliability.
 - ❖ Dashboards and reports provide useful insights for better decision making and planning.
 - ❖ The system reduces human error and simplifies complex inventory workflows.
- ❖ Although features like offline mode, notifications and supplier integration were not implemented, the system provides a strong foundation for future expansion.
- ❖ The project also strengthened practical skills in Flutter development, database design, system integration and problem solving.



Thank you